Final Project - Progress Presentation II Slide

Understanding and Improving GPT-2 Implementations in NanoGPT

Wang Zeyu

National Taiwan University

Outline

Introduction

This presentation:

- Connects NanoGPT's multi-head attention implementation to the mathematical notation taught in lectures.
- Explains differences between code and slides.
- Summarises code-level observations.
- Outlines next steps for implementing **Key–Value (KV) caching** to NanoGPT to improve inference efficiency.

Task 1

Connecting Multi-head Attention Implementation in NanoGPT

Notation Recap

We adopt the slide notation and map it to NanoGPT:

- B: batch size
- T: sequence length
- d: embedding dimension (`n_embd`)
- h: number of attention heads
- $d_h = d/h$: head dimension or embedding dimension per head
- $\tilde{Z} \in \mathbb{R}^{T \times d}$: input matrix (single sequence)
- $x \in \mathbb{R}^{B \times T \times d}$: input tensor (batched) in NanoGPT implementation

Note: Highlighted terms indicate differences from lecture slides.

Single-Head Attention (Lecture)

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

with

$$Q = \tilde{Z} W_Q, \quad K = \tilde{Z} W_K, \quad V = \tilde{Z} W_V$$

Single-Head Attention (Lecture)

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

with

$$Q = \tilde{Z} W_Q, \quad K = \tilde{Z} W_K, \quad V = \tilde{Z} W_V$$

For multi-head attention:

$$\text{head}_i = \text{Softmax}\left(\frac{\tilde{Z} W_Q^{(i)} (\tilde{Z} W_K^{(i)})^\top}{\sqrt{d_h}}\right) \tilde{Z} W_V^{(i)}$$

$$\text{MultiHead}(\tilde{Z}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) W_O$$

Difference 1: Combined Projections

Instead of 3 different weights each, NanoGPT combines W_Q, W_K, W_V into one linear projection:

$$W_{\text{combined}} = [W_Q | W_K | W_V] \in \mathbb{R}^{d \times 3d}$$

and

$$b_{\text{combined}} = [b_Q | b_K | b_V] \in \mathbb{R}^{3d}$$

Difference 1: Combined Projections

Instead of 3 different weights each, NanoGPT combines W_Q, W_K, W_V into one linear projection:

$$W_{\text{combined}} = [W_Q | W_K | W_V] \in \mathbb{R}^{d \times 3d}$$

and

$$b_{\text{combined}} = [b_Q | b_K | b_V] \in \mathbb{R}^{3d}$$

Then, using x (batched input) instead of $\tilde{Z}$ (single sequence input):

$$x \in \mathbb{R}^{B \times T \times d}, \quad \text{out} = x \cdot W_{\text{combined}} + b_{\text{combined}}$$

$$\text{out} \in \mathbb{R}^{B \times T \times 3d}$$

How is this matrix multiplication possible?

$$\text{out} = x \cdot W_{\text{combined}} + b_{\text{combined}}$$

$$x \in \mathbb{R}^{B \times T \times d}, W_{\text{combined}} \in \mathbb{R}^{d \times 3d}, b_{\text{combined}} \in \mathbb{R}^{3d}$$

- $3D \times 2D + 1D$ absolutely breaks the logic of linear algebra.

How is this matrix multiplication possible?

$$\text{out} = x \cdot W_{\text{combined}} + b_{\text{combined}}$$

$$x \in \mathbb{R}^{B \times T \times d}, W_{\text{combined}} \in \mathbb{R}^{d \times 3d}, b_{\text{combined}} \in \mathbb{R}^{3d}$$

- $3D \times 2D + 1D$ absolutely breaks the logic of linear algebra.
- However, PyTorch supports **batched matrix multiplication and broadcasting**.
- W is multiplied over x across all the $(B \times T)$ positions, resulting in a $\mathbb{R}^{B \times T \times 3d}$.
- Then, b is broadcasted over all the $(B \times T \times d)$ positions.
- This is why operations across different dimensions are possible in PyTorch.

Reason for Combination for batched multiplication

Using PyTorch's **batched matrix multiplication + broadcasting**:

- Avoids sequential computation of Q, K, and V.
- Enables **GPU-parallel execution**.
- More memory efficient and less computation time.

Reason for Combination for batched multiplication

Using PyTorch's **batched matrix multiplication + broadcasting**:

- Avoids sequential computation of Q, K, and V.
- Enables **GPU-parallel execution**.
- More memory efficient and less computation time.

To get back the Q, K, V, we simply split the output into three parts along the last dimension:

$$Q, K, V = \text{split}(\text{out};\ d, d, d)$$

$$Q = xW_Q + b_Q, \quad K = xW_K + b_K, \quad V = xW_V + b_V$$

$$\text{where } Q, K, V \in \mathbb{R}^{B \times T \times d}$$

Difference 2: Head Splitting and Reshaping

After getting $Q, K, V \in \mathbb{R}^{B \times T \times d}$, NanoGPT also reshapes and transposes them. Here, we use Q as example:

Reshape:

$$Q \in \mathbb{R}^{B \times T \times d} \longrightarrow Q' \in \mathbb{R}^{B \times T \times h \times d_h}$$

Transpose:

$$Q' \longrightarrow Q'' \in \mathbb{R}^{B \times h \times T \times d_h}$$

Thus,

$$Q'', K'', V'' \in \mathbb{R}^{B \times h \times T \times d_h}$$

Difference 2: Head Splitting and Reshaping

After getting $Q, K, V \in \mathbb{R}^{B \times T \times d}$, NanoGPT also reshapes and transposes them. Here, we use Q as example:

Reshape:

$$Q \in \mathbb{R}^{B \times T \times d} \longrightarrow Q' \in \mathbb{R}^{B \times T \times h \times d_h}$$

Transpose:

$$Q' \longrightarrow Q'' \in \mathbb{R}^{B \times h \times T \times d_h}$$

Thus,

$$Q'', K'', V'' \in \mathbb{R}^{B \times h \times T \times d_h}$$

Each head i:

$$Q''^{(i)}, K''^{(i)}, V''^{(i)} \in \mathbb{R}^{B \times T \times d_h}$$

Reason for Reshaping and Transposing

Without Reshaping and Transposing:

$$\text{for } i = 1, \ldots, h: \quad \text{head}_i = \text{Softmax}\left(\frac{Q^{(i)}(K^{(i)})^\top}{\sqrt{d_h}}\right) V^{(i)}.$$

each head i is calculated sequentially in a loop.

Reason for Reshaping and Transposing

Without Reshaping and Transposing:

$$\text{for } i = 1, \ldots, h: \quad \text{head}_i = \text{Softmax}\left(\frac{Q^{(i)}(K^{(i)})^\top}{\sqrt{d_h}}\right) V^{(i)}.$$

each head i is calculated sequentially in a loop.

After reshaping, each head is processed in parallel:

$$\text{MultiHead}(x) = \text{head}_{\text{all}} = \text{Softmax}\left(\frac{Q''(K'')^\top}{\sqrt{d_h}}\right) V''$$

PyTorch interprets (B, h, T, d_h) as $B \cdot h$ independent "batches" of size (T, d_h), batched matrix multiplication.
Reorganises so that all heads can be computed in parallel using a single batched matrix multiplication.
$\rightarrow$ computed efficiently in a **single GPU kernel call**.

Ending Notes for Task 1

After looking through the implementation in NanoGPT, we realise

- Mathematical definition **doesn't change**
- Simply **reorganises** so that everything can be computed in **parallel**
- Ultimately, its so that its more computationally **efficient** and less GPU-intensive

Task 2

Implement and Evaluate KV Caching

KV Caching in GPT-2 (Reference)

Let's try to reference from GPT-2's `model.py` to get an idea of how to do KV caching in NanoGPT.

KV Caching in GPT-2 (Reference)

Let's try to reference from GPT-2's `model.py` to get an idea of how to do KV caching in NanoGPT.

Idea: Reuse past key/value pairs to avoid recomputation. Using GPT-2 as reference:

```
def attn(x, scope, n_state, *, past, hparams):
    assert x.shape.ndims == 3
    assert n_state % hparams.n_head == 0
    if past is not None:
        assert past.shape.ndims == 5
```

GPT-2 uses a `past` argument, which is defined as below:

$$\text{past} = (\boldsymbol{k}_1, \ldots, \boldsymbol{k}_{T-1}, \ \boldsymbol{v}_1, \ldots, \boldsymbol{v}_{T-1})$$

KV Caching in GPT-2 (Reference)

```
def attn(x, scope, n_state, *, past, hparams):
...
        if past is not None:
            pk, pv = tf.unstack(past, axis=1)
            k = tf.concat([pk, k], axis=-2)
            v = tf.concat([pv, v], axis=-2)
```

KV Caching in GPT-2 (Reference)

```
def attn(x, scope, n_state, *, past, hparams):
...
        if past is not None:
            pk, pv = tf.unstack(past, axis=1)
            k = tf.concat([pk, k], axis=-2)
            v = tf.concat([pv, v], axis=-2)
```

past is split into $\boldsymbol{k}_1, \ldots, \boldsymbol{k}_{T-1}$ and $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_{T-1}$
Let's simplify this as k_{past} and v_{past} respectively.

KV Caching in GPT-2 (Reference)

```
def attn(x, scope, n_state, *, past, hparams):
...
        if past is not None:
            pk, pv = tf.unstack(past, axis=1)
            k = tf.concat([pk, k], axis=-2)
            v = tf.concat([pv, v], axis=-2)
```

past is split into $\boldsymbol{k}_1, \ldots, \boldsymbol{k}_{T-1}$ and $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_{T-1}$
Let's simplify this as k_{past} and v_{past} respectively.
The current key/value pair will now be:

$$k = [k_{\text{past}}; k_{\text{new}}], \quad v = [v_{\text{past}}; v_{\text{new}}]$$

This speeds up autoregressive text generation because only the newest token's attention is recomputed.

Next Steps

Modify `CausalSelfAttention.forward`:

1. Add optional `past` argument for (k, v) cache.
2. Concatenate cached and new tensors along sequence dimension.
3. Update `sample.py` to feed cached tensors during inference.
4. Benchmark inference time and memory usage before/after.

Thank You

Questions?